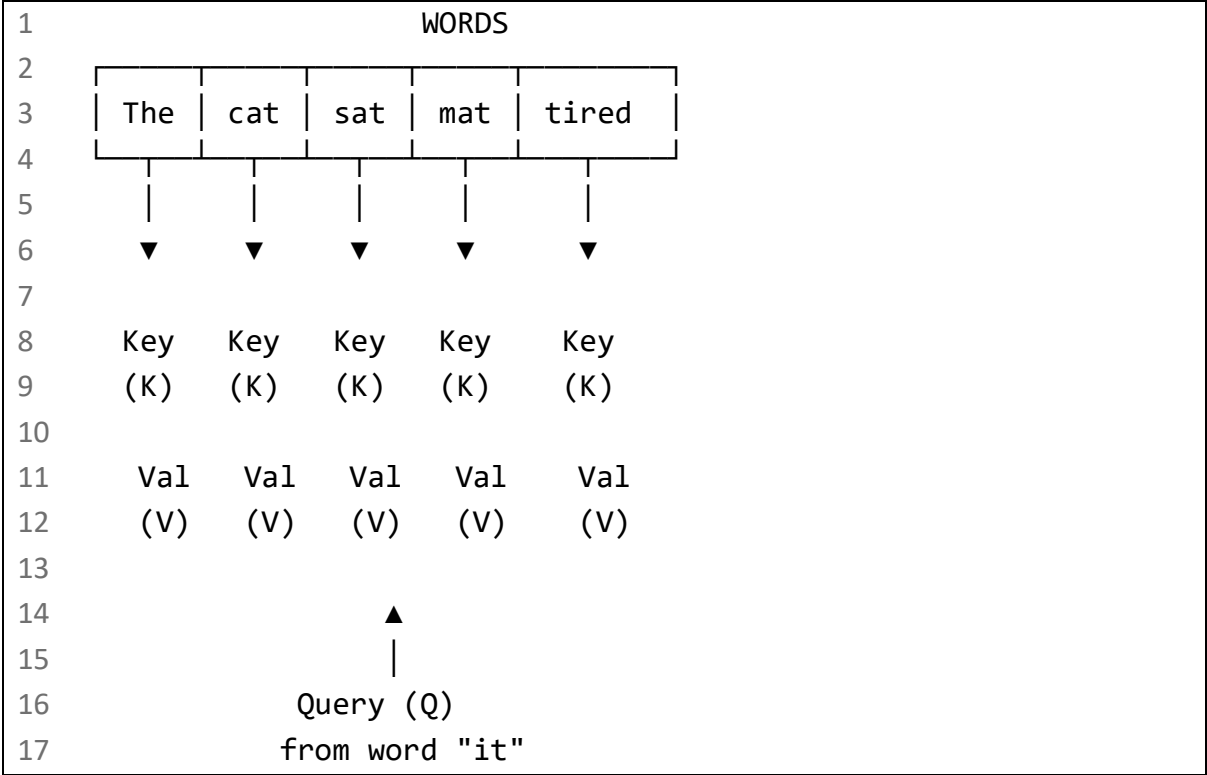


Absolutely. Here's a simple visual of what happens when the word "it" attends to the sentence:

"The cat sat on the mat because it was tired"

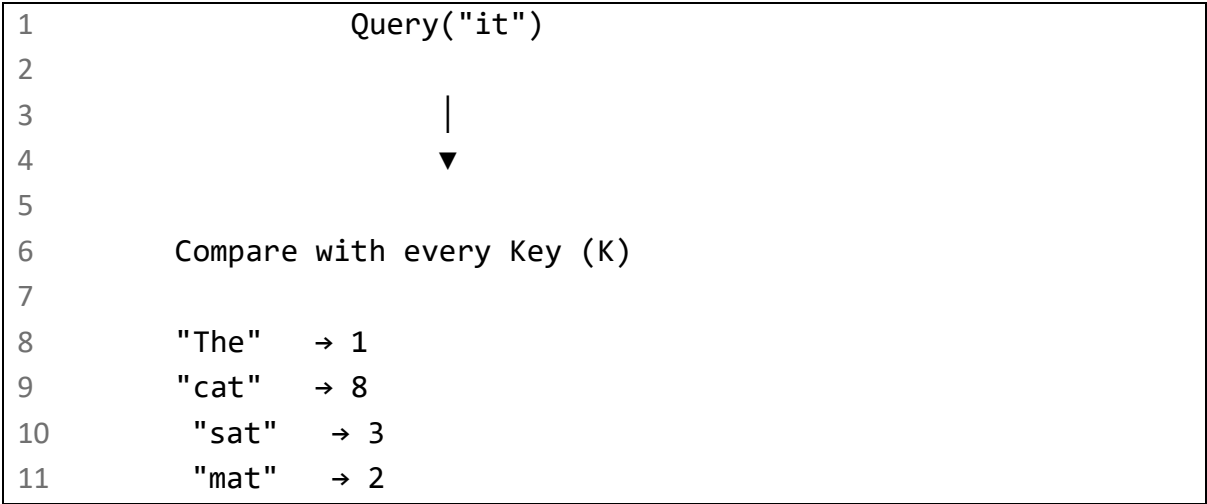
Step 1: Create Query, Keys, and Values



The Query asks:

"Which previous word helps me understand 'it'?"

Step 2: Compare Query with Every Key



```
12      "tired" → 4
13
14  Result:
15
16   $QK^T = [1, 8, 3, 2, 4]$ 
```

Think of these as **relevance scores**.

Step 3: Scale the Scores

```
1   $QK^T / \sqrt{dk}$ 
2
3  Assume  $dk = 64$ 
4
5   $\sqrt{64} = 8$ 
6
7   $[1, 8, 3, 2, 4]$ 
8       $\div 8$ 
9
10  $= [0.125, 1.0, 0.375, 0.25, 0.5]$ 
```

This simply prevents values from becoming too large.

Step 4: Softmax

Convert scores into percentages.

```
1  softmax(
2  [0.125, 1.0, 0.375, 0.25, 0.5]
3  )
```

```
4
```

```
5  ↓
```

```
6
```

```
7
```

8	Word	Weight
9		
10	The	11%
11	cat	36%
12	sat	15%
13	mat	13%
14	tired	25%
15		

Visualized:

1	The	■
2	cat	■ ■ ■ ■ ■ ■ ■ ■
3	sat	■ ■ ■
4	mat	■ ■ ■
5	tired	■ ■ ■ ■ ■

The model says:

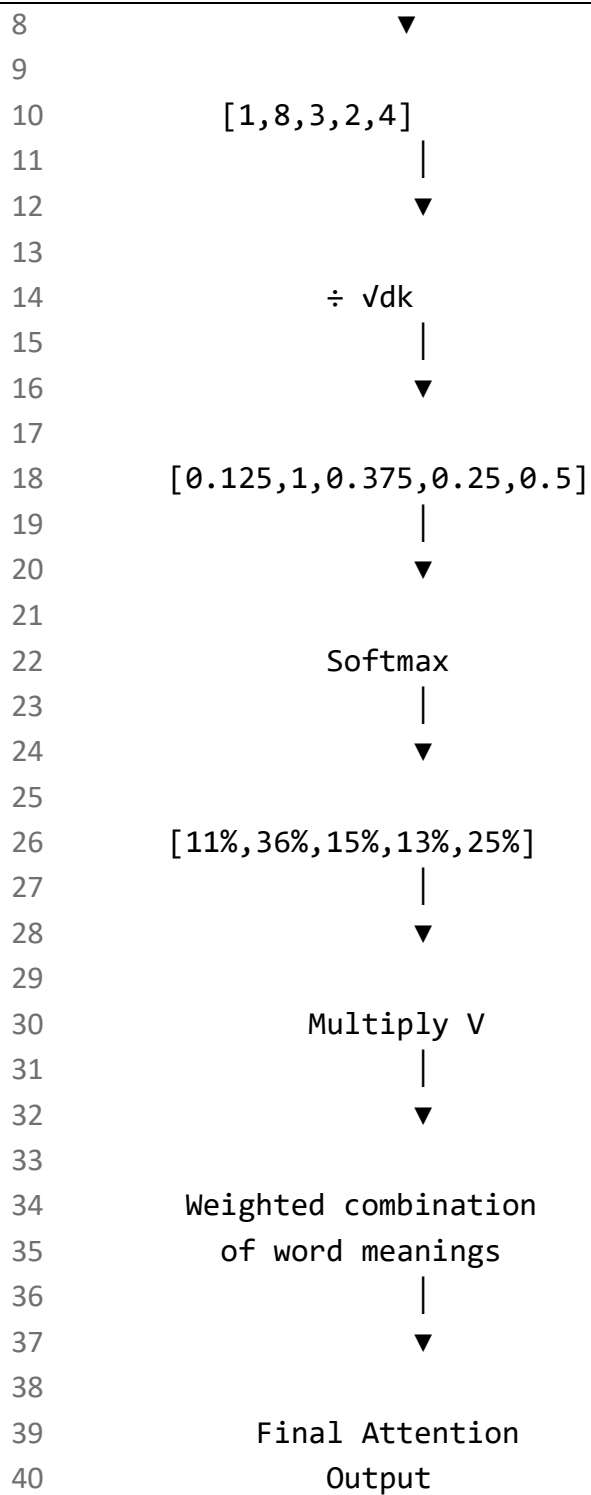
"Pay most attention to **cat**."

Step 5: Use Those Weights on the Values

1	Attention Weights
2	
3	11% → V(The)
4	36% → V(cat)
5	15% → V(sat)
6	13% → V(mat)
7	25% → V(tired)
8	
9	
10	▼
11	
12	Weighted Combination
13	
14	
15	▼
16	
17	New representation
18	for the word "it"

The Entire Pipeline

1	Query
2	Q
3	
4	▼
5	
6	$Q \times K^T$ Similarity
7	



A nice mental model is:

- 1 Q = What am I looking for?
- 2 K = What information do I contain?
- 3 V = The actual information
- 4
- 5 Q vs K → Who matters?
- 6 Softmax → How much do they matter?

7 $\times V \rightarrow$ Gather their information.

That's the entire transformer attention mechanism in one picture. The rest of GPT is essentially applying this process many times, in many layers, and with many attention heads operating in parallel.